

User Manual Symphony

Version 1.0

Eslab

March 14, 2025

Contents

1	Introduction	5
2	Background	5
2.1	Radiation experiments	5
2.2	Symphony	5
2.2.1	Architecture	6
2.2.2	Device Under Test	7
3	Getting started	8
3.1	Requirement installation	8
4	Interacting with Symphony	9
4.1	JSON	9
4.2	Interacting with the JSON	11
4.3	Load from JSON	11
4.3.1	Function signature	11
4.3.2	Description	11
4.3.3	Parameters	11
4.3.4	Returns	11
4.3.5	Usage Example	12
4.4	Load from Dictionary	12
4.4.1	Function signature	12
4.4.2	Description	12
4.4.3	Parameters	12
4.4.4	Returns	12
4.4.5	Usage Example	12
5	Build-in useful functions	13
5.1	Find minimum operating voltage	13
5.1.1	Function signature	13
5.1.2	Parameters	13
5.1.3	Returns	13
5.1.4	Usage Example	14
5.2	Experiment start	14
5.2.1	Function signature	14
5.2.2	Description	15
5.2.3	Parameters	15

5.2.4	Returns	15
5.2.5	Usage Example	15
5.3	Undervolt experiment	15
5.3.1	Function signature	16
5.3.2	Description	16
5.3.3	Parameters	16
5.3.4	Returns	16
5.3.5	Usage Example	16
6	General purpose functions	17
6.1	power_handler	17
6.1.1	Function signature	17
6.1.2	Description	17
6.1.3	Parameters	17
6.1.4	Returns	17
6.1.5	Usage Example	18
6.2	remote_alive	18
6.2.1	Function signature	18
6.2.2	Description	18
6.2.3	Parameters	18
6.2.4	Returns	19
6.2.5	Usage Example	19
6.3	remote_execute	19
6.3.1	Function signature	19
6.3.2	Description	19
6.3.3	Parameters	19
6.3.4	Returns	20
6.3.5	Usage Example	20
6.4	simple_remote_execute	20
6.4.1	Function signature	20
6.4.2	Description	21
6.4.3	Parameters	21
6.4.4	Returns	21
6.4.5	Usage Example	21
6.5	set_callback	22
6.5.1	Function signature	22
6.5.2	Description	22
6.5.3	Parameters	22

6.5.4	Returns	22
6.5.5	Usage Example	22
7	User defined callbacks	23
7.1	actions_on_reboot	23
7.1.1	Callback signature	23
7.1.2	Description	23
7.1.3	Parameters	23
7.1.4	Returns	23
7.1.5	Usage Example	23
7.2	additional_logs	24
7.2.1	Callback signature	24
7.2.2	Description	24
7.2.3	Parameters	24
7.2.4	Returns	24
7.2.5	Usage Example	25
7.3	detect_cache_upsets	25
7.3.1	Callback signature	25
7.3.2	Description	25
7.3.3	Parameters	25
7.3.4	Returns	25
7.3.5	Usage Example	25
7.4	is_result_correct	26
7.4.1	Callback signature	26
7.4.2	Description	26
7.4.3	Parameters	26
7.4.4	Returns	26
7.4.5	Usage Example	27
7.5	dut_monitor	27
7.5.1	Callback signature	27
7.5.2	Description	27
7.5.3	Parameters	27
7.5.4	Returns	27
7.5.5	Usage Example	28
7.6	dut_health_check	28
7.6.1	Callback signature	28
7.6.2	Description	29
7.6.3	Parameters	29

7.6.4	Returns	29
7.6.5	Usage Example	29
7.7	request_voltage_value	30
7.7.1	Callback signature	30
7.7.2	Description	30
7.7.3	Parameters	30
7.7.4	Returns	30
7.7.5	Usage Example	30
7.8	target_power_button	30
7.8.1	Callback signature	30
7.8.2	Description	31
7.8.3	Parameters	31
7.8.4	Returns	31
7.8.5	Usage Example	31
7.9	target_reset_button	31
7.9.1	Callback signature	31
7.9.2	Description	31
7.9.3	Parameters	32
7.9.4	Returns	32
7.9.5	Usage Example	32
7.10	undervolt_format	32
7.10.1	Callback signature	32
7.10.2	Description	32
7.10.3	Parameters	32
7.10.4	Returns	33
7.10.5	Usage Example	33
7.11	update_all	33
7.11.1	Callback signature	33
7.11.2	Description	33
7.11.3	Parameters	34
7.11.4	Returns	34
7.11.5	Usage Example	34
8	Expanding the framework	34
8.1	Inheritance Model	34

1 Introduction

The provided user manual is associated with a Python-based framework called **Symphony**, which is utilized for conducting radiation-based experiments. Difficulty and safety during such experiments are the most significant factors; thus, Symphony is designed to prevent human interference, using the network infrastructure and various micro-controllers, like **Raspberry Pi**, to control situations that otherwise would demand humans to interfere.

The forenamed framework uses the **client-server** architecture to acquire the necessary features. This architecture typically uses two distinct systems, one that plays the role of the server side and the other that plays the role of the client side. Both the server and client, according to Symphony's terminology, can be referred to with a unique name. The server side can be called **Device Under Test (DUT)**, while the Client side is the **Host**.

2 Background

2.1 Radiation experiments

Radiation experiments made in specialized facilities, where a neutron-radiation accelerator may lie, are used to evaluate the reliability of computer systems such as **FPGAs**, **CPUs**, and **ASICs** in an environment of high radiation. The most common use of the data gained from such experiments is to strengthen systems to survive in space.

For a radiation experiment to succeed, some prerequisites are required. The most important of them is that people mustn't enter the room where charged particles are present. To accomplish this, specialized software must be made to autonomously control the systems under test inside the accelerator's room. Continuing in the current chapter, we will introduce a framework that is used to ensure both the safety of people and the integrity of the data gained in an experiment in which high radiation is present.

2.2 Symphony

As stated in previous chapters, Symphony constitutes a framework leveraged for conducting radiation experiments. To accomplish this, Symphony

lets users customize the necessities required for an experiment. The framework provides the user with an API to instruct Symphony to handle certain crucial situations.

The Symphony's API provides the following basic functionalities for radiation experiments:

- A way to define the benchmarks to execute during the experiment
- A way to handle crucial situations, such as kernel panic, crashes, and more (as depicted [Table 1](#)).
- Give the user the freedom to choose what to do in the most common situations expected in such an experiment (through some callbacks, which will be covered later).
- Saves the results in easy process JSON files.

Situation	Handle
Kernel Panic	Restart
PL Crash	Restart
SDCs	LOG
Netowrk Error	LOG
System Error (recovered)	LOG

Table 1: Crusial situations handled by Symphony

2.2.1 Architecture

Symphony's architecture uses the Client-Server network model. As the model suggests, it uses the network infrastructure to establish a connection between two systems, one located inside the radiation room, called Device Under Test (DUT), and the other that controls the DUT device from outside, called Host. The Host is responsible for the client-side operations, while the DUT is responsible for the server side.

The architecture of Symphony can be depicted in [Figure 1](#). As illustrated in [Figure 1](#), the Host and **DUT** are connected using a gateway router. Another detail spotted in the figure is an entity called "LOGIC" and "CALLBACK" both clarified later in this sentence. Logic constitutes hardware responsible for performing a hard reset on the DUT system. Callback, if we overuse the original terminology, it can be called a "driver". This callback instructs Symphony to

perform a hard reset, when necessary, using the so-called "LOGIC" hardware in between.

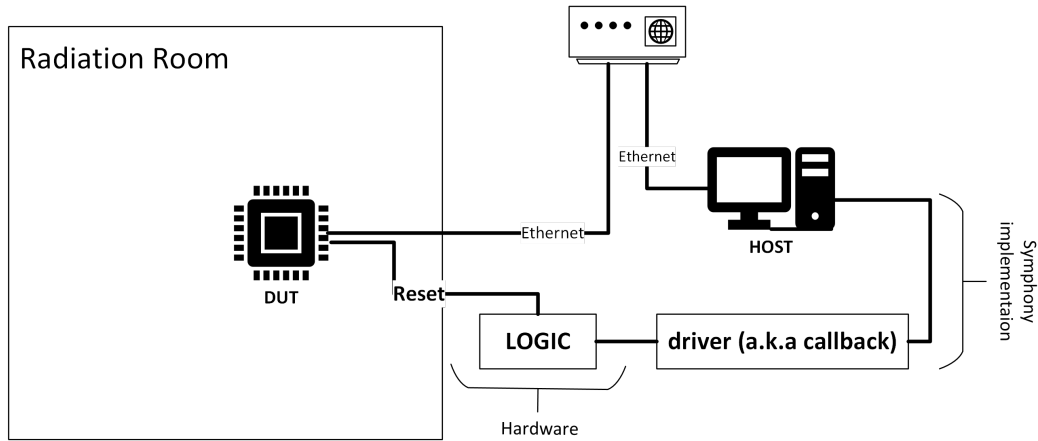


Figure 1: Symphony's Architecture

The driver (a.k.a. callback) is user-defined and varies between implementations. Section 7 describes how the callback is defined and how to implement such a routine.

2.2.2 Device Under Test

As depicted in Figure 2, the DUT is located inside the experiment room. This device is responsible for executing any command requested from the Host. Another task is to send back, through the network infrastructure, the results of the executed commands. The message that contains the result is in a specified dictionary format, shown in Figure 2.

```
{
    "healthlog": ... ,
    "run_command": ... ,
    "timestamp": ... ,
    "duration_ms": ... ,
    "stdoutoutput": ... ,
    "stderr": ... ,
    "return_code": ... ,
    "dmesg_diff": ... ,
}
```

Figure 2: DUT's result message

3 Getting started

3.1 Requirement installation

To install Symphony, follow these steps: Clone the repository and navigate to the symphony directory::

```
git clone git@github.com:unipieslab/symphony.git
cd symphony
```

Next, depending on your Linux package manager, execute the appropriate script:

For RedHat-based Linux, run:

```
chmod +x install-Python<VERSION>_dnf.sh
./install-Python<VERSION>_dnf.sh
```

For Debian-based Linux, run:

```
chmod +x install-Python<VERSION>_apt.sh
./install-Python<VERSION>_apt.sh
```

Once these steps are successfully completed, Symphony and all its dependencies will be installed. Lastly, to prepare the environment for Symphony to run, navigate to the following directory:

```
| cd host
```

Then, execute the following command:

```
| make
```

After completing this command, the host component of Symphony's client-server architecture can be started by executing the following script:

```
| ./runHost.sh
```

It will then immediately attempt to connect to the DUT, if the component is up and running.

4 Interacting with Symphony

4.1 JSON

To start using **Symphony**, the user must provide certain pieces of information for the framework to start conducting the experiment. This includes a benchmark list to be executed, an experiment ID (to differentiate between results), the expected duration of the entire experiment, and other details as shown in [Table 2](#).

The information mentioned before is defined via **JSON** format file; containing the respective fields for those listed in [Table 2](#). Not all fields listed in [Table 2](#) are mandatory for Symphony's operation, some fields are used in specific scenarios, such as undervolting, while others have predefined values. The user can distinguish such fields from [Table 2](#) by looking at **column 2 (Importance)**.

Caution is required for the fields specified as "undervolted" in column 2 of [Table 2](#). Information related to undervolting can be ignored if the experiment is not undervolt-related. The respective fields in the JSON file, if not used, must remain empty, either by setting as value **"null"** or **"[]"** or **"{}"** depending on the type of each field.

There could be cases where the experiment requires more information than those listed in [Table 2](#). In such cases, the user can define additional fields in the JSON file to satisfy the requirements. Should the user add additional

fields to the JSON file, then the respective code must be written in order to handle the extra information, see the expanding framework section.

JSON Entry	Importance	defaults
effective_time_per_batch_s	OPTIONAL	20
finish_after_total_effective_min	OPTIONAL	100
finish_after_total_errors	OPTIONAL	100
voltage_commands	UNDERVOLT	-
benchmark_commands	REQUIRED	-
timeouts	REQUIRED	-
voltage_list	UNDERVOLT	-
benchmark_list	REQUIRED	-
target_ip	REQUIRED	-
target_port	REQUIRED	-
setup_id	REQUIRED	-

Table 2: Required json entries

4.2 Interacting with the JSON

As mentioned, the user can interact with **Symphony** via a **JSON**, by storing the desired values in the fields listed in [Table 2](#). After the filing process, the Host parses the JSON file given by the user as input and translates it into an internal dictionary, used to conduct the experiment, based on the input information. Parsing of the JSON is performed only once.

Following in this section the user will get an idea of how to use Symphony to read the corresponding JSON file of interest.

4.3 Load from JSON

4.3.1 Function signature

```
| def load_experiment_attr_from_json_file(self, src: str)
```

4.3.2 Description

```
| Read the contents of the JSON, specified in src,  
| and store them in an internal dictionary.
```

4.3.3 Parameters

```
| src: The JSON file from which the entries are to be loaded
```

4.3.4 Returns

```
| Nothing is returned
```

4.3.5 Usage Example

```
tester = Tester_Shell() # An instance of Symphony (Host)

json_file = "example.json" # The JSON that holds the
    entries
tester.load_experiment_attr_from_json_file(json_file)
```

4.4 Load from Dictionary

4.4.1 Function signature

```
def load_experiment_attr_from_dict(self, src: dict)
```

4.4.2 Description

Read the contents of a **Dicitionary**, specified in **src**, and store them in an internal dictionary.

4.4.3 Parameters

src: The Dictionary from which the entries are to be loaded

4.4.4 Returns

Nothing is returned

4.4.5 Usage Example

```
tester = Tester_Shell() # An instance of Symphony (Host)

entries = {
    "effective_time_per_batch_s": null,
    "finish_after_total_effective_min": null,
    "finish_after_total_errors": null,
    ... # The rest of the entries
} # The dictionary that holds the entries
tester.load_experiment_attr_from_dict(entries)
```

5 Build-in useful functions

There are repeatedly implemented cases in a radiation-based experiment where at least one device is exposed to a source of high ionizing radiation. Such scenarios, for instance, are to monitor the devices that are being tested and to handle operations such as resetting a crashed device due to high radiation. For this reason, Symphony has implemented some built-in functions that could accelerate the development process.

Following this section, the user will learn about the available built-in functions.

5.1 Find minimum operating voltage

An essential part of an undervolting experiment is to find, through experimenting, a voltage value that is the minimum voltage in which the device being undervolted is operating correctly. To save time, Symphony has a built-in function that does exactly that.

5.1.1 Function signature

```
def auto_undervolt_characterization(self,  
    duration_per_bench_min: int, characterization_id: str)  
    -> int
```

5.1.2 Parameters

duration_per_bench_min: An integer representing the number of minutes each benchmark should be on a specific (undervolted) voltage value.

characterization_id: This field represents a string that corresponds to an identification for the test.

5.1.3 Returns

Typically returns an integer representing the value of the requested voltage (aka Vmin). Otherwise, nothing is

returned, and the user must examine the logs to determine Vmin.

5.1.4 Usage Example

```
tester = Tester_Shell() # An instance of Symphony (Host)
undervolt_cmd= "Bash_command_for_undervolting"
v_nominal = N # Where N is the initial voltage upon boot (
    in hexadecimal)

res = tester.auto_undervolt_characterization(n_nominal,
    undervolt_cmd) # Find the Minimum operating voltage
```

(To run the above example, there are a few preliminary steps to follow. Refer to [section 3](#))

5.2 Experiment start

A monitoring routine that examines the behavior of a device is a complex and time-consuming procedure to develop. Such a routine must be able to retrieve integral information such as the following:

- The temperature of the system.
- The current-voltage value (on undervolt experiments).
- Determine the overall health of the system
- Possible SDCs that occurred

Moreover, such a routine must take action in scenarios where human interference is otherwise required. One such scenario is if the device had a kernel panic and must reset to continue the experiment.

To relieve the user from developing a routine like this, Symphony provides a built-in function that considers all the mentioned scenarios and more.

5.2.1 Function signature

```
def experiment_start(self)
```

5.2.2 Description

Monitors the DUT system for any possible issues, take action when necessary, and save JSON formatted files that contain data from DUT (for post-processing and analysis).

This routine, however, requires several user-implemented functions to perform with the expected behavior (see section with callbacks).

5.2.3 Parameters

No parameters are required.

5.2.4 Returns

Nothing is returned

5.2.5 Usage Example

```
tester = Tester_Shell() # An instance of Symphony (Host)

# User-defined functions...
# Code to make adjustments to the DUT system ...

tester.experiment_start()
```

5.3 Undervolt experiment

Another routine that is often vital is a routine that performs an undervolting experiment. Such a routine must take care of a few essential actions listed below:

- To reduce the voltage of the system.
- Evaluate the device's overall health in an (undervolted) voltage value.
- Execute each benchmark for the instructed total time.

Symphony has an existing implementation of such a routine, explained in the following definition.

5.3.1 Function signature

```
def target_perform_undervolt_test(self)
```

5.3.2 Description

Performs an undervolting experiment for some minutes, specified in the JSON. This routine makes the undervolting process and benchmark execution easy and automatic.

This routine, however, requires several user-implemented functions to perform with the expected behavior (see section with callbacks).

5.3.3 Parameters

No parameters are required.

5.3.4 Returns

Nothing is returned

5.3.5 Usage Example

```
tester = Tester_Shell() # An instance of Symphony (Host)
# User-defined functions...
# Code to make adjustments to the DUT system ...

tester.target_perform_undervolt_test()
```

6 General purpose functions

6.1 power_handler

6.1.1 Function signature

```
def power_handler(self, action: Tester_Shell_Power_Action)
```

6.1.2 Description

Is responsible for shutting/resetting down the DUT system when necessary. The User may use this routine to implement custom procedures.

Although the way each system shuts down or resets is almost the same, there is a variety of ways to reset a system in an experiment. One way is to use Raspberry PI's or UART modules combined with a relay to accomplish these two actions. Because of this variance, this routine requires a user-defined routine that explicitly instructs how the shutdown/reset procedures can be accomplished (See section `set_callback`).

6.1.3 Parameters

action: Can receive two possible actions, either to simulate the press of the power button or to simulate the press of the reset button. To accomplish the press of the power button, the value **TARGET_POWER_BTN_PRESS** must be passed on the routine. To accomplish the press of the reset button, the value **TARGET_RESET_BTN_PRESS** must be passed on the routine.

6.1.4 Returns

Nothing is returned

6.1.5 Usage Example

```
tester = Tester_Shell() # An instance of Symphony (Host)

tester.power_handler(Tester_Shell_Power_Action.
    TARGET_POWER_BTN_PRES) # Simulate the action of
    pressing the power button on DUT.

tester.power_handler(Tester_Shell_Power_Action.
    TARGET_RESET_BTN_PRESS) # Reset (reboot) the DUT system
```

6.2 remote_alive

6.2.1 Function signature

```
def remote_alive(self, net_timeout_s: int, ret_immediate:
    bool) -> bool
```

6.2.2 Description

Check whether the DUT system is down. In case of any (communication-related) error that may encountered during the execution of the requested command, there will be attempts to finish the execution. If three attempts of execution have been attempted, then, the DUT is ordered to do a hard reset.

6.2.3 Parameters

net_timeout_s: Represents the expected delay due to the network infrastructure in seconds. Users may choose their value of preference, but Symphony offers the built-in constant value **NETWORK_TIMEOUT_SEC**, which can be used instead.

ret_imidiatae: Represents a logical value that allows the users to force the routine to return immediately after encountering any (communication-related) error.

6.2.4 Returns

Either **True**, in case the DUT is up, or **False** in case it is down.

6.2.5 Usage Example

```
tester = Tester_Shell() # An instance of Symphony (Host)

tester.remote_alive(Tester_Shell_Constants.
    NETWORK_TIMEOUT_SEC.value, False) # Check whether the
    DUT system is visible on the network.
```

6.3 remote_execute

6.3.1 Function signature

```
def remote_execute(self, cmd: str, cmd_timeout_s: int,
    net_timeout_s: int, dmesg_index: int, times: int,
    ret_immediate: bool) -> list:
```

6.3.2 Description

Executes user-requested bash commands to the DUT system. In case of any (communication-related) error that may encountered during the execution of the requested command, there will be attempts to finish the execution. If three attempts of execution have been attempted, then, the DUT is ordered to do a hard reset.

6.3.3 Parameters

cmd: Represents a string that is the command to execute
cmd_timeout_s: An integer representing the expected number of seconds it takes from DUT to execute the requested command. Timeouts like this can be estimated using the routine "estimate_timeouts" (see section)
net_timeout_s: Represents the expected delay due to the

network infrastructure in seconds. Users may choose their value of preference, but Symphony offers the built-in constant value **NETWORK_TIMEOUT_SEC**, which can be used instead.

dmesg_index: This parameter represents an integer that specifies the position where the last command left the dmesg file. The user may use this function with **dmesg = 0**, as the genuine usage of this parameter matters only in the internal implementation of the symphony.

times: An integer representing the number of times the DUT system must execute the requested command.

ret_immediate: Represents a logical value that allows the users to force the routine to return immediately after encountering any (communication-related) error.

6.3.4 Returns

Returns a list holding the results of the requested command. The list contains an element number equal to the number of times the requested command was executed.

6.3.5 Usage Example

```
tester = Tester_Shell() # An instance of Symphony (Host)
ls_cmd_timeout = 2 # seconds

tester.remote_execute("ls", ls_cmd_timeout,
    Tester_Shell_Constants. NETWORK_TIMEOUT_SEC.value, 0,
    2, False)
```

6.4 simple_remote_execute

6.4.1 Function signature

```
def simple_remote_execute(self, cmd: str, times: int,  
    ret_immediate: bool) -> list
```

6.4.2 Description

Is a simpler version of the corresponding `remote_execute` routine. The only difference between these two routines is that the `simple_remote_execute` routine requires fewer parameters than `remote_execute` does, making it easier to use in some cases where the extra parameters are redundant, like executing a command whose execution time is only a few seconds. If three attempts of execution have been attempted, then, the DUT is ordered to do a hard reset

6.4.3 Parameters

cmd: Represents a string that is the command to execute

times: An integer representing the number of times the DUT system must execute the requested command.

ret_immediate: Represents a logical value that allows the users to force the routine to return immediately after encountering any (communication-related) error.

6.4.4 Returns

Returns a list holding the results of the requested command. The list contains an element number equal to the number of times the requested command was executed

6.4.5 Usage Example

```
tester = Tester_Shell() # An instance of Symphony (Host)  
ls_cmd_timeout = 2 # seconds
```

```
tester.simple_remote_execute("ls", 2, False)
```

6.5 set_callback

6.5.1 Function signature

```
def set_callback(self, callback_func, callback_id:  
    Tester_Shell_Callback) -> None
```

6.5.2 Description

Assigns a user-defined routine to an internal routine used for actions like resetting or monitoring the DUT. These actions can't be implemented in a way that works in every implementation and DUT system. Thus, the user must implement it for the specific scenario at hand.

6.5.3 Parameters

callback_func: Represents a function pointer pointing to the user-defined function of interest.

callback_id: Specifies which internal routine to replace with the user-defined routine (see section for the list of internal routines available for modification).

6.5.4 Returns

Nothing is returned

6.5.5 Usage Example

```
def reset_dut():  
    # Code that performs hard reset.
```

```

tester = Tester_Shell() # An instance of Symphony (Host)
ls_cmd_timeout = 2 # seconds

# How does the dut perform a hard reset?
tester.set_callback(reset_dut, Tester_Shell_Callback.
    TARGET_RESET_BUTTON)

# Start the experiment.
tester.experiment_start()

```

7 User defined callbacks

7.1 actions_on_reboot

7.1.1 Callback signature

```
def __callback_actions_on_reboot(self) -> None
```

7.1.2 Description

Specifies actions that must be performed when the DUT system reboots. For example, it can restore the voltage stage to where it was before rebooting.

7.1.3 Parameters

No parameters are required.

7.1.4 Returns

Nothing is returned

7.1.5 Usage Example

```
curr_voltage = 850 # mV
```

```

def actions_on_reboot_user_defined():
    # code ...
    global curr_voltage
    restore_voltage(curr_voltage)

tester = Tester_Shell() # An instance of Symphony (Host)
tester.set_callback(
    actions_on_reboot_user_defined, # User defined
    function
    Tester_Shell_Callback.ACTIONS_ON_REBOOT # The
    coresponded internal callback
)

```

7.2 additional_logs

7.2.1 Callback signature

```

def __callback_additional_logs(self) -> str

```

7.2.2 Description

When Symphony's `experiment_start` method is called, except that it controls the DUT system, it logs important information about the duration of the experiment and the DUT system. To discourage users from modifying the internal logs, this callback is being used instead to allow the user to add additional logs if they desire.

7.2.3 Parameters

No parameters are required.

7.2.4 Returns

A string representing the desired additional logs should be returned.

7.2.5 Usage Example

```
def my_additional_logs() -> str:
    temp    = get_temp()
    voltage = get_voltage()
    return "TEMP(C):" + str(temp) + "Voltage_(mV):" + str(
        voltage)

tester = Tester_Shell() # An instance of Symphony (Host)
tester.set_callback(my_additional_logs,
```

7.3 detect_cache_upsets

7.3.1 Callback signature

```
def __callback_detect_cache_upsets(self, dmesg: str) ->
    None
```

7.3.2 Description

Is used to specify a way to detect cache upsets that may be present in the DUT system.

7.3.3 Parameters

dmesg: The current portion of DMESG contents.

7.3.4 Returns

Nothing is returned

7.3.5 Usage Example

```
class My_tester(Tester_Shell): # Sub-class of Tester_Shell
    (see section 8)
    def __init__(self):
```

```

        self.total_cache_upsets = 0

    def upset_detect_user_defined(dmesg: str) -> None:
        # Analyze the dmesg...
        self.total_cache_upsets += new_upsets

tester = My_tester() # An instance of Symphony (Host)
tester.set_callback(
    tester.upset_dectect_user_defined, # User-defined
    method
    Tester_Shell_Callback.DETECT_CACHE_UPSETS
)

```

7.4 is_result_correct

7.4.1 Callback signature

```

def __callback_is_result_correct(self, result: dict) ->
    bool

```

7.4.2 Description

Checks whether a result is correct. Users may implement other mechanisms to handle a wrong result in addition to the check when a result is not considered correct.

7.4.3 Parameters

result: The currently examined result.

7.4.4 Returns

Returns **True** if the examined result is considered correct, otherwise **False** is returned.

7.4.5 Usage Example

```
def is_result_correct(result: dict):
    # Analyze the result...
    if (correct == True):
        return True
    else:
        return False

tester = Tester_Shell() # An instance of Symphony (Host)

tester.set_callback(
    is_result_correct, # User-defined
    Tester_Shell_Callback.IS_RESULT_CORRECT
)
```

7.5 dut_monitor

7.5.1 Callback signature

```
def __callback_dut_monitor(self, healthlog: str) -> None
```

7.5.2 Description

This callback can be used to monitor the DUT system's resources, for example, to retrieve the temperature. Is possible to use this callback in combination with other callbacks, as depicted in the following example.

7.5.3 Parameters

healthlog: Constitute a string that contains crucial health-related information transferred from the DUT system (also user-defined)

7.5.4 Returns

No parameters are required.

7.5.5 Usage Example

```
class My_tester(Tester_Shell): # Sub-class of Tester_Shell
    (see section 8)
    def __init__(self):
        self.temperature = 0

    # Retrieve the temperature from DUT.
    def monitor_dut_user_defined(self) -> None:
        # Retrieve the temperature from DUT system...
        self.temperature = dut_current_temp

    # Log the temperature of DUT.
    def my_additional_logs_user_defined(self) -> str:
        return "TEMP(C):" + str(self.temperature)

tester = My_tester() # An instance of Symphony (Host)
# Define the instructions of how to perform health-check
tester.set_callback(
    tester.monitor_dut_user_defined, # User-defined method
    Tester_Shell_Callback.DUT_MONITOR
)

# Define additional logs.
tester.set_callback(
    tester.my_additional_logs_user_defined, # User-defined
    method
    Tester_Shell_Callback.ADDITIONAL_LOGS.
)
```

7.6 dut_health_check

7.6.1 Callback signature

```
def __callback_dut_health_check(self, tester: Tester_Shell
    ) -> str
```

7.6.2 Description

Evaluates the health of the DUT system. It is an important callback used in the internal implementation of Symphony to determine when the DUT must be reset.

7.6.3 Parameters

tester: The instance of Symphony (**Host**). User may use every forenamed build-in function to determine the health of the DUT system.

7.6.4 Returns

returns an instance of the `Tester_Shell_Health_Status`. If DUT is healthy, **`Tester_Shell_Health_Status.HEALTHY`** must be returned, otherwise **`Tester_Shell_Health_Status.DAMAGED`** must be returned.

7.6.5 Usage Example

```
def check_health(tester: Tester_Shell):  
    # Determine if the DUT is healthy.  
    if (not healthy):  
        return Tester_Shell_Health_Status.DAMAGED  
  
    return Tester_Shell_Health_Status.HEALTHY  
  
tester = Tester_Shell() # An instance of Symphony (Host)  
  
tester.set_callback(  
    check_health, # user-defined  
    Tester_Shell_Callback.DUT_HEALTH_CHECK  
)
```

7.7 request_voltage_value

7.7.1 Callback signature

```
def __callback_unvervolt_format(self) -> str
```

7.7.2 Description

Must return the current voltage value of the regulator in the DUT system.

7.7.3 Parameters

No parameters are required.

7.7.4 Returns

A string representing the voltage value

7.7.5 Usage Example

```
# For undervolting process.
current_voltage = 850 # mV

def voltage_value_user_defined() -> str
    # Code that retrives the voltage value from DUT...
    return voltage_value

tester = Tester_Shell() # An instance of Symphony (Host)
tester.set_callback(
    voltage_value_user_defined,
    Tester_Shell_Callback.REQUEST_VOLTAGE_VALUE
)
```

7.8 target_power_button

7.8.1 Callback signature

```
|def __callback_target_power_button(self) -> None
```

7.8.2 Description

|This callback must be defined in such a way as to simulate the press of the power button.

7.8.3 Parameters

|No parameters are required.

7.8.4 Returns

|Nothing is returned

7.8.5 Usage Example

```
|def power_button() -> None:
|    # Code to simulate the press of the power button...

|tester = Tester_Shell() # An instance of Symphony (Host)
|tester.set_callback(
|    power_button,
|    Tester_Shell_Callback.TARGET_POWER_BUTTON
|)
```

7.9 target_reset_button

7.9.1 Callback signature

```
|def __callback_target_reset_button(self) -> None
```

7.9.2 Description

|This callback must be defined in such a way as to simulate the press of the reset button.

7.9.3 Parameters

No parameters are required.

7.9.4 Returns

Nothing is returned

7.9.5 Usage Example

```
def reset_button() -> None:
    # Code to simulate the press of the power button...

tester = Tester_Shell() # An instance of Symphony (Host)
tester.set_callback(
    reset_button,
    Tester_Shell_Callback.TARGET_RESET_BUTTON
)
```

7.10 undervolt_format

7.10.1 Callback signature

```
def __callback_unvervolt_format(self) -> str
```

7.10.2 Description

This callback is exclusively used for undervolt characterization, where the Symphony discovers the minimum voltage value in which DUT is still operating. The task assigned for this callback is to give Symphony a format that can be used in each step of the undervolting process to reduce the voltage of the DUT system.

7.10.3 Parameters

No parameters are required.

7.10.4 Returns

A string representing the next voltage value

7.10.5 Usage Example

```
# For undervolting process.
current_voltage = 850 # mV

def undervolt_user_defined() -> str
    global current_voltage
    disired_step = 10 # mV
    command = "aproprate_command_to_reduce_voltage" +
        curr_voltage
    curr_voltage = curr_voltage - disired_step

    return command

tester = Tester_Shell() # An instance of Symphony (Host)
tester.set_callback(
    undervolt_user_defined,
    Tester_Shell_Callback.UNDERVOLT_FORMAT
)

test.auto_undervolt_characterization(0.10, "UNDERVOLT_1")
```

7.11 update_all

7.11.1 Callback signature

```
def __callback_update_all(self) -> None
```

7.11.2 Description

Symphony updates various variables during an experiment to keep track of its state. The user can use this callback to define their variables to update along with the

internal variables. Variables are updated when there is a voltage value or a benchmark.

7.11.3 Parameters

No parameters are required.

7.11.4 Returns

No parameters are required.

7.11.5 Usage Example

```
tester = Tester_Shell() # An instance of Symphony (Host)

tester.power_handler(Tester_Shell_Power_Action.
    TARGET_POWER_BTN_PRES) # Simulate the action of
    pressing the power button on DUT.

tester.power_handler(Tester_Shell_Power_Action.
    TARGET_RESET_BTN_PRESS) # Reset (reboot) the DUT system
```

8 Expanding the framework

Symphony is designed to create an abstraction between the user and the internal implementation. This abstraction comes at the cost of immobilizing the user from editing anything from the internal implementation. Internal functionalities may not be able to handle certain scenarios, though. For such scenarios, the user can use the inheritance model that Python delivers to expand the framework's functionalities to handle new scenarios. This chapter will describe in detail how to expand this framework.

8.1 Inheritance Model

Like any object-oriented programming language, Python has an inheritance model that allows the developer to create a subclass from another class.

In the case of Symphony, the developer must inherit from the **Tester_Shell** class to expand the framework's implementation. Below is an illustrative example that will clarify any issue.

```
class My_Tester(Tester_Shell):
    def __init__(self):
        super().__init__()
        # class Variables...

    def reset_button() -> None: # Method used for callback
        .
        # Code to simulate the press of the power button..

    def new_functionality_1(self):
        # Code...

    # Methods...

tester = My_Tester() # An instance of the new implementation
tester.set_callback(
    tester.reset_button,
    Tester_Shell_Callback.TARGET_RESET_BUTTON
)

tester.new_functionality_1()
tester.start_experiment()
```

It is important to note that the sub-class's init method must always call the **super.init()** method in order for the parent class, **Tester_Shell**, to initialize its internal components. Moreover, another advantage of this implementation is that the developer can implement their callbacks inside the subclass and reference them, as shown in the previous code, to assign the corresponding callback to an internal one, using the **set_callback** method ([subsection 6.5](#)).